ACTIVITY NO. 1

REVIEW OF C++ PROGRAMMING	
Course Code: CPE010	**Program:** Computer Engineering
Course Title: Data Structures and Algorithms	**Date Performed: 10/07/2026**
Section: CPE21S4	**Date Submitted: 10/07/2026**
Name: Seth Carlo T. Miraflor	**Instructor: Engr. Jimlord Quejado**

1. Objective(s)

- Implement basic programming and OOP in C++

2. Intended Learning Outcomes (ILOs)

After this module, the student should be able to:

a. Create code that follows the basic C++ code structure;
b. Implement appropriate class definition and instances based on given requirements;
c. Solve different problems using the C++ programming language.

3. Discussion

Part A: Introduction to C++ Code

Structure of C++ Code

Sections	**Sample Code**
Header File Declaration Section	`#include<iostream>` `using namespace std;`
Global Declaration Section	`int count = 0;`

Class Declaration and Method Definition Section	`class rectangle{`
	`private:`
	`  double recLength, recWidth;`
	
	`public:`
	`  rectangle(double L, double W);`
	`  void setLength(double L);`
	`  void setWidth(double W);`
	`  double getPerimeter();`
	`};`

Main Function	`int main(){`
	`  rectangle shape1(2, 5);`
	`  std::cout << "The perimeter of the rectangle is " << shape1.getPerimeter() << ".\n";`
	`  std::cout << count << " number of objects created.";`
	`  return 0;`
	`}`

Method Definition	`rectangle::rectangle(double L, double W) {`
	`  recLength = L;`
	`  recWidth = W;`
	`  count++;`
	`}`

```
void rectangle::setLength(double L) {
    recLength = L;
}

void rectangle::setWidth(double W) {
    recWidth = W;
}

double rectangle::getPerimeter() {
    return (2*recLength) + (2*recWidth);
}
```

It is not required for all sections to have code for every use-case. However, for best practices you would prefer to have an overall structure to follow to increase code readability and reusability.

Data Types

d. Primary Data Type: int, float, char and void
e. User defined data type: structure, union, class, enumeration
f. Derived data type: array, function, pointer, reference

Local & Global Variables

```
#include <iostream>
using namespace std;

int globalVal = 0; //Global Variable

int main(){
    int localVal = 5; //Local Variable

    std::cout << "Global Variable has value " << globalVal << ".\n";
    std::cout << "Local Variable has value " << localVal << ".\n";

    return 0;
}
```

Operators

Arithmetic	Relational	Logical
Addition +	Greater than >	AND &&
Subtraction –	Less than <	OR ||
Multiplication *	Greater than or equal >=	NOT !
Division /	Less than or equal <=	
Modulo %	Equal ==	
Increment ++	Not equal !=	
Decrement --		

Bitwise Operators

Let A = 60 and B = 13. Binary values are as follows:

```
A = 0011 1100
B = 0000 1101
```

Bitwise AND -> &	`A & B`	`0000 1100`
Bitwise OR -> |	`A | B`	`0011 1101`
Bitwise XOR -> ^	`A ^ B`	`0011 0001`
Bitwise Complement -> ~	`~A`	`1100 0011`

Assignment Operator

Assign a value to a variable. Example:

Assign the value 20 to a variable A.

```
int A = 20;
```

The assignment operator is a basic component denoted as "=".

Part B: Classes and Objects using C++

To create a class use the class keyword. Syntax is:

```
class myClass {
  public:
      int myNum;
      string myString;
};
```

`public` here is an access specifier. It indicates that the attributes and methods listed under it are accessible outside the class. A simple table is provided below to summarize the access specifiers used in c++.

Specifiers	Within same class	In derived class	Outside the class
`private`	Yes	No	No
`protected`	Yes	Yes	No
`public`	Yes	Yes	Yes

We can then create an object from this class:

```
int main(){
      //this creates the object
      myClass object1;

      //this accesses the public attributes
      object1.myNum = 5;
      object1.myString = "Sample";

      return 0;
}
```

4. Materials and Equipment

Personal Computer with C++ IDE

Recommended IDE:

- CLion (must use TIP email to download)
- DevC++ (use the embarcadero fork or configure to C++17)

5. Procedure

ILO A: Create Code That Follows the Basic C++ Code Structure

For this activity, you have to demonstrate the use of a **function prototype**. The section on class declaration and method definition will be used for the function prototype and the function will be defined in the follow method definition section after the main function.

A function prototype in c++ is a declaration of the name, parameters and return type of the function before its definition. Write a C++ code the satisfies the following:

- Create a function that will take two numbers and display the sum.
- Create a function that will return whether variable A is greater than variable B.
- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

Note:

- The driver program must call each function.
- The definitions must be after the main function.

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

In this section, the initial implementation for a class **triangle** will be implemented. The step-by-step procedure is shown below:

Step 1. Include the necessary header files. For this one, we only need `#include <iostream>`

Step 2. Create the triangle class. Assign it with private variables: totalAngle, angleA, angleB, and angleC.

```
class Triangle{
private:
    double totalAngle, angleA, angleB, angleC;
```

Step 3. We then create public methods. The constructor must allow for creation of the object with 3 initial angles to be stored in our previously defined variables `angleA, angleB` and `angleC`. Another method has to be made if the user wants to change the initial values, this will also accept 3 arguments to change the values in `angleA, angleB` and `angleC`. Lastly, a function to validate whether the given values make our shape an actual triangle.

```
public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
};
```

Step 4. Define the methods.

```
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
```

```
        angleB = B;
        angleC = C;
        totalAngle = A+B+C;
    }

    void Triangle::setAngles(double A, double B, double C) {
        angleA = A;
        angleB = B;
        angleC = C;
        totalAngle = A+B+C;
    }

    const bool Triangle::validateTriangle() {
        return (totalAngle <= 180);
    }
```

Step 5. Create the driver code.

```
    int main(){
        //driver code
        Triangle set1(40, 30, 110);
        if(set1.validateTriangle()){
            std::cout << "The shape is a valid triangle.\n";
        } else {
            std::cout << "The shape is NOT a valid triangle.\n";
        }

        return 0;
    }
```

Include the output of running this code in section 6. Note your observations and comments.

6. Output

Sections	Answer
Header File Declaration Section	`#include <iostream>` `using namespace std;`
Global Declaration Section	None
Class Declaration and Method Definition Section	class Triangle
Main Function	cpp int main()
Method Definition	cpp Triangle::Triangle

Table 1-1. C++ Structure Code for Answer

```
 const bool Triangle::validateTriangle() {
    return (angleA > 0 && angleB > 0 && angleC > 0 &&
            totalAngle == 180);
}
```

Observation:

- The program successfully creates a `Triangle` object using the constructor.
- The constructor stores the three angle values (40, 30, and 110).

- The `validateTriangle()` method computes the total angle (180°) and returns `true`.
- The `main()` function displays **"The shape is a valid triangle."**

Comment:

- The program demonstrates the use of a class, private data members, a constructor, member functions, object instantiation, and method calls.
- A more accurate triangle validation should also ensure that **each angle is greater than 0** and that **the sum of the three angles is exactly 180°**, not merely less than or equal to 180°. A better validation function would be:

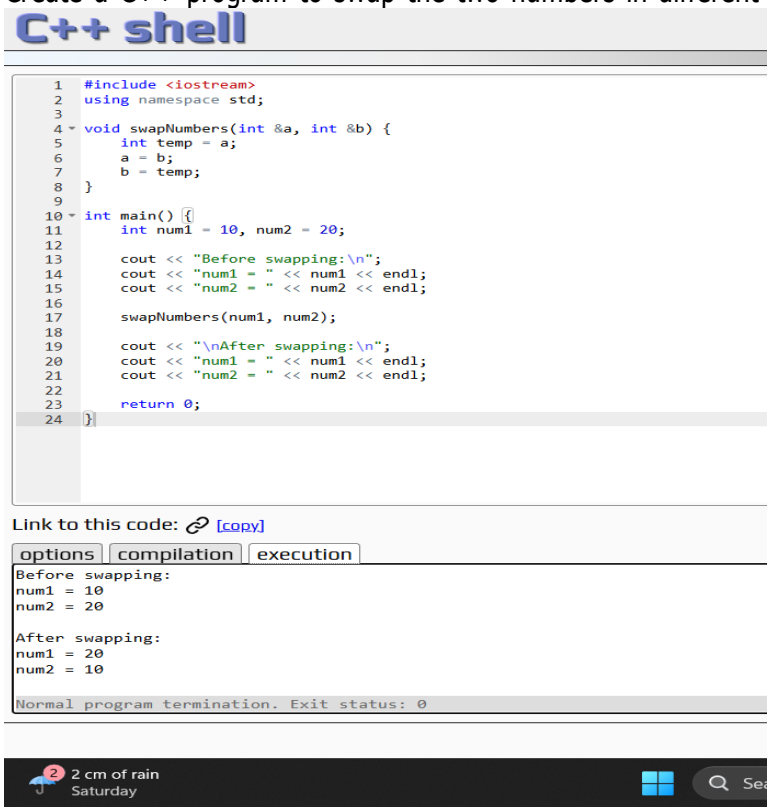
Table 1-2. ILO B output observations and comments.

7. Supplementary Activity

ILO C: Solve Different Problems using the C++ Programming Language

The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.



2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

C++ shell

```cpp
#include <iostream>
using namespace std;

double kelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9 / 5 + 32;
}

int main() {
    double kelvin;

    cout << "Enter temperature in Kelvin: ";
    cin >> kelvin;

    cout << "Temperature in Fahrenheit: "
         << kelvinToFahrenheit(kelvin) << endl;

    return 0;
}
```

Link to this code: [copy]

options | compilation | execution

```
Enter temperature in Kelvin: 25
Temperature in Fahrenheit: -414.67

Normal program termination. Exit status: 0
```



3. Create a C++ program that has a function that will calculate the distance between two points.

C++ shell

```cpp
#include <iostream>
#include <cmath>
using namespace std;

double distance(double x1, double y1, double x2, double y2) {
    return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}

int main() {
    double x1, y1, x2, y2;

    cout << "Enter x1 y1: ";
    cin >> x1 >> y1;

    cout << "Enter x2 y2: ";
    cin >> x2 >> y2;

    cout << "Distance = "
         << distance(x1, y1, x2, y2);

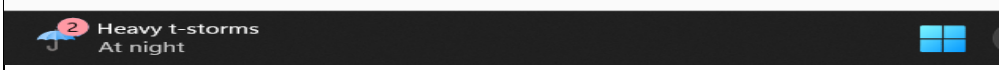
    return 0;
}
```

Link to this code: [copy]

options | compilation | execution

```
Enter x1 y1: y1
Enter x2 y2: Distance = 0

Normal program termination. Exit status: 0
```



4. Modify the code given in ILO B and add the following functions:
    a. A function to compute for the area of a triangle

```
double computeArea() {
    double s = (sideA + sideB + sideC) / 2;
    return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}
```

    b. A function to compute for the perimeter of a triangle

```
double computePerimeter() {
    return sideA + sideB + sideC;
}
```

    c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```
string triangleType() {
    if (angleA < 90 && angleB < 90 && angleC < 90)
        return "Acute Triangle";
    else if (angleA == 90 || angleB == 90 || angleC == 90)
        return "Right Triangle";
    else if (angleA > 90 || angleB > 90 || angleC > 90)
        return "Obtuse Triangle";
    else
        return "Others";
}
```

Provide the following:

- Summary of lessons learned
    - This activity enhanced my understanding of the basic structure of a C++ program, including header files, function prototypes, classes, objects, constructors, and member functions. I also learned how to solve programming problems using functions and object-oriented programming concepts.
- Analysis of the procedure
    - The procedure provided a step-by-step guide in creating functions and classes. It helped me understand how to separate class declarations, method definitions, and the main function, making the program more organized and easier to maintain.
- Analysis of the supplementary activity
    - The supplementary activities strengthened my programming skills by applying functions to solve different problems such as swapping variables, converting temperatures, calculating distances, and extending a class with additional methods. These exercises improved my understanding of functions, classes, and object-oriented programming.
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?
    - I believe I completed the activity successfully because I was able to create programs that met the given requirements and apply the concepts discussed in the lesson. My areas for improvement include gaining more experience in designing classes, debugging programs efficiently.

9. Assessment Rubric